

Université de Lille
Faculté des Sciences et Technologies
Cité Scientifique, Avenue Henri Poincaré
59655 Villeneuve-d'Ascq, France

INRIA Lille – Nord Europe
Parc Scientifique de la Haute Borne
Équipe EVREF

Licence 3 Informatique

Année Universitaire 2025 – 2026

RAPPORT DE STAGE INTERMÉDIAIRE

Évolution et optimisation d'un framework de Property-Based Testing pour Pharo

Période du stage :

du 13 avril au 28 août 2026

Rédigé et présenté par :
Stella-Rose MILLE

Tuteur en Entreprise :
Federico LOCHBAUM

Tuteur Universitaire :
Romain ROUVOY

Table des matières

1	Contexte du Stage	2
1.1	Présentation du Laboratoire et de l'Équipe	2
1.2	Contexte de l'Application : Le projet Ume	2
2	Description d'une Réalisation Technique Majeure	3
2.1	Contexte et Objectif du Shrinking	3
2.2	Choix Techniques et Justifications	3
2.3	Développements et Travaux Réalisés	3
2.4	Difficultés rencontrées et Solutions apportées	3
3	Conclusion et Bilan du Stage	5
3.1	Adéquation de la Réalisation aux Objectifs et Perspectives	5
3.2	Présentation du Travail à Réaliser pour la Suite	5
3.3	Enseignements et Apports du Stage	5
3.4	Remerciements	5

1. Contexte du Stage

1.1 Présentation du Laboratoire et de l'Équipe

Ce stage se déroule au sein d'INRIA (Institut National de Recherche en Sciences et Technologies du Numérique), un organisme public de recherche dédié aux sciences du numérique. Plus précisément, j'ai intégré l'équipe de recherche Evref.

Leur principal objectif est l'étude de l'évolution des grands systèmes informatiques. Leurs travaux s'articulent autour de trois axes : L'analyse et la migration, le développement d'outils, l'infrastructure.

1.2 Contexte de l'Application : Le projet Ume

Mon travail s'inscrit dans le cadre d'un projet de thèse de l'équipe, centré sur le développement d'Ume. Ume est un framework de Property-Based Testing (PBT) conçu spécifiquement pour l'écosystème Pharo (un langage purement orienté objet dérivé de Smalltalk). Contrairement aux tests unitaires classiques où le développeur écrit manuellement les entrées, le PBT génère automatiquement des centaines d'entrées aléatoires pour valider des propriétés globales du logiciel. Ume cherche à pousser ce concept plus loin en l'appliquant à la détection de régressions et d'anomalies de performance.

2. Description d'une Réalisation Technique Majeure

Au cours de ces sept premières semaines de stage, j'ai réalisé plusieurs améliorations centrées sur l'API d'Ume afin de simplifier son adoption par des utilisateurs extérieurs. J'ai notamment fluidifié la définition d'une campagne de test et implémenté une barre de progression multithreadée pour offrir un retour visuel en temps réel lors de l'exécution. Cependant, ma réalisation majeure réside dans l'intégration du mécanisme de shrinking.

2.1 Contexte et Objectif du Shrinking

Le shrinking intervient lorsqu'un test échoue sur une entrée complexe générée aléatoirement. L'objectif est de réduire cet input au cas minimal reproduisant l'erreur. Bien que l'application du shrinking aux problématiques pures de performance reste un sujet de recherche ouvert, cette fonctionnalité est indispensable pour permettre à l'utilisateur de localiser l'origine précise d'un bug fonctionnel sans être noyé sous des données superflues.

2.2 Choix Techniques et Justifications

L'implémentation du shrinking dans Ume s'inspire des travaux de référence du Fuzzing Book. Mon premier choix s'est porté sur la création d'un algorithme de Delta Debugging. Basé sur le principe du « diviser pour régner », cet algorithme segmente la chaîne de caractères d'entrée et évalue chaque morceau pour isoler la faille.

Cependant, le Delta Debugging classique montre ses limites lorsqu'il doit manipuler des entrées structurées. Si la réduction casse la syntaxe attendue par le programme, le test échoue pour une mauvaise raison (erreur de parsing). Pour résoudre cette problématique, nous avons choisi de coupler cette approche à des **réducteurs basés sur la grammaire**, garantissant que chaque entrée réduite reste syntaxiquement valide.

2.3 Développements et Travaux Réalisés

Pour mener à bien cette réalisation, j'ai adopté une méthodologie rigoureuse de développement piloté par les tests (TDD). Cela m'a permis de définir précisément le comportement attendu de mes réducteurs avant d'écrire le code métier. J'ai mis en place le design pattern Strategy pour l'architecture du shrinker. Ce choix permet d'isoler les différents algorithmes de réduction (Delta Debugging, réduction de grammaire) au sein de classes distinctes, rendant le framework facilement extensible pour de futurs algorithmes.

2.4 Difficultés rencontrées et Solutions apportées

- **Manque de documentation et de détails théoriques :** Les articles académiques détaillent rarement l'implémentation bas niveau des algorithmes. J'ai dû analyser des codebases open-source existants pour en comprendre la logique fine.
- **Adaptation au paradigme Smalltalk/Pharo :** Les exemples de référence (notamment du *Fuzzing Book*) sont écrits en Python. Transposer une logique impérative ou procédurale vers le modèle purement orienté objet de Pharo a demandé une réingénierie complète des structures de données.

- **Solidité de l'exploration** : Garantir que le shrinker explore efficacement les branches de réduction sans boucler indéfiniment a nécessité la mise en place d'une suite de tests aux limites très stricte.

3. Conclusion et Bilan du Stage

3.1 Adéquation de la Réalisation aux Objectifs et Perspectives

Les objectifs de cette première moitié de stage ont été atteints : l'API d'Ume a été simplifiée pour l'utilisateur final et une première brique fonctionnelle de shrinking respectant la grammaire a été intégrée. Concernant la question de l'évaluation, la mise en place de **benchmarks** précis constitue l'étape logique suivante. Ils permettront de mesurer l'impact du shrinking sur le temps de calcul et le nombre de tests générés.

3.2 Présentation du Travail à Réaliser pour la Suite

La suite de mes missions s'articulera autour de deux axes :

- L'implémentation complète et l'optimisation des algorithmes de réduction de grammaire existants.
- La réalisation de campagnes de Benchmarks pour quantifier les performances du shrinker.
- L'objectif à terme est de concevoir un système capable d'aiguiller intelligemment la campagne de shrinking vers l'algorithme le plus adapté au cas par cas, afin de limiter le coût computationnel actuel de cette procédure.

3.3 Enseignements et Apports du Stage

Ce stage m'a permis de développer une forte autonomie. Dès les premiers jours, mon tuteur a mis l'accent sur l'auto-organisation. J'ai appris à chercher l'information par moi-même en lisant des articles de recherche. Nous avons structuré notre flux de travail avec GitHub pour la gestion de versions (apprentissage des Pull Requests professionnelles et bien documentées) et **Obsidian** pour la documentation de mes tâches quotidiennes.

Sur le plan technique, l'utilisation rigoureuse du TDD m'a permis de canaliser mes développements lorsque je perdais de vue l'objectif final, tandis que l'application concrète des Design Patterns a renforcé mes compétences en conception logicielle.

3.4 Remerciements

Je souhaite remercier mon tuteur d'entreprise, Federico Lochbaum, pour ses précieux conseils, sa patience et sa confiance. Mon tuteur universitaire, Romain Rouvoy, pour son suivi rigoureux. Enfin, j'adresse mes remerciements à Imen Sayar pour son soutien continu et pour avoir appuyé ma candidature pour ce stage.